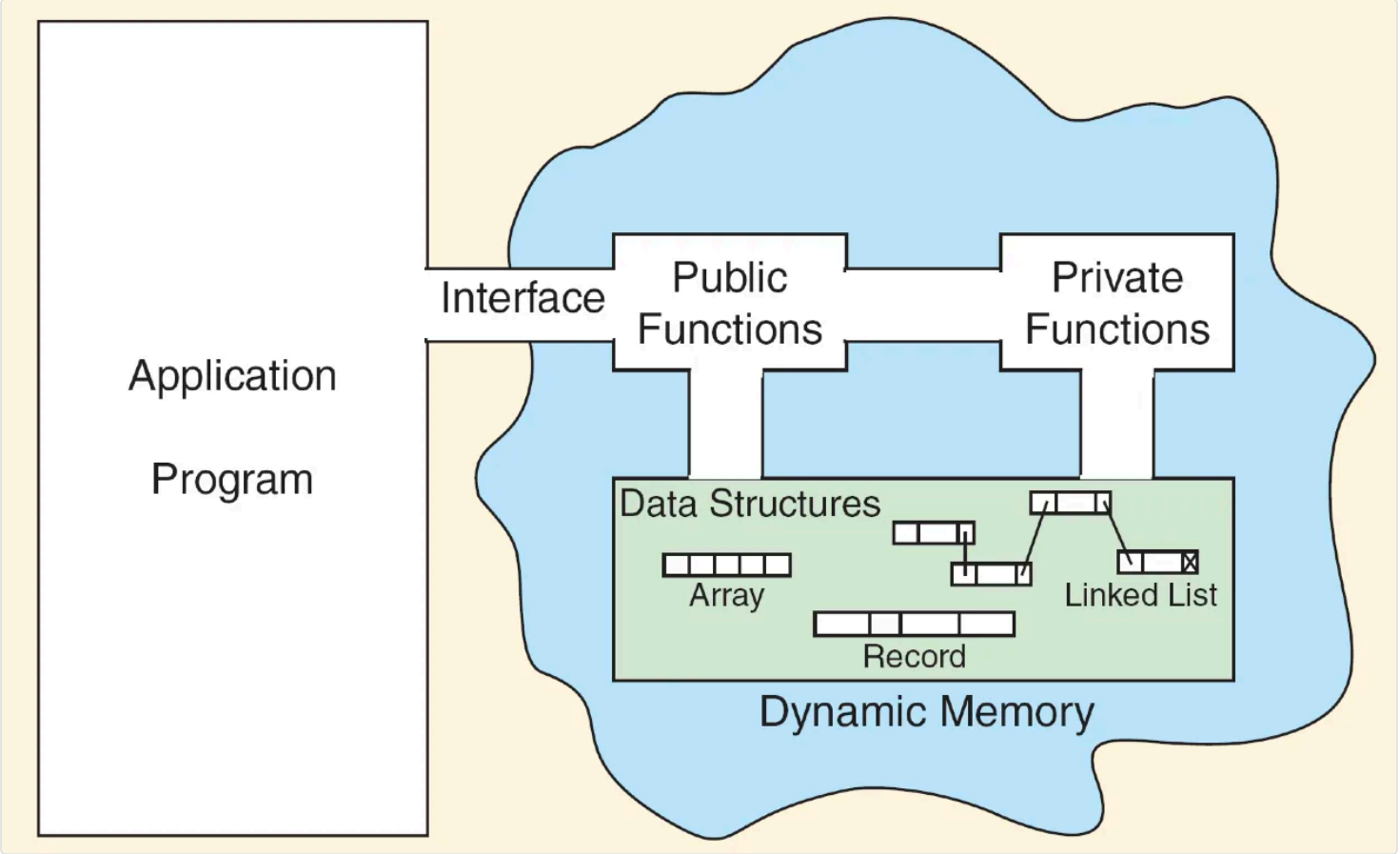
Lecture 10 · Object-Oriented Programming(객체지향 프로그래밍)

강의 · Notion 아카이브

Abstract Data Type(추상 데이터 타입, ADT)

- 

- '어떤 데이터 타입은 ~ **Object(객체)**, ~ **Function(함수)**로 구성되어 있다' ⇒ 사용설명서
- 구현 부분을 나타내지 않음 → 비공개적
- 사용법만 알면 ⇒ 데이터 타입을 쉽게 쓸 수 있음
- ADT를 통해 데이터나 연산이 무엇(what)인지는 알 수 있지만, 어떻게(how) 동작하는지는 알 수 없음
- 즉, 데이터에 대한 연산들의 **set(집합)**을 묶어서 하나의 단위로 정의하는 프로그래밍 개념
  - ⇒ 프로그래밍 디자인과 구현을 용이하게 함

    Encapsulation(캡슐화)
    - 객체지향프로그래밍
    - 연관된 데이터와 함수를 논리적으로 묶어놓음, 데이터를 보호하기 위해 다른 객체의 접근을 제한 ⇒ **Class(클래스)** 명령어를 사용하여 구현할 수 있음

Object Oriented Programming (객체 지향 프로그래밍)

- 기능을 묶어두는 것이 아니라, 데이터와 처리하는 기능을 묶은 하나의 단일 프로그램 객체
- **Abstraction(추상화)**

  - 복잡한 현실 세계를 객체로 모델링 → 단순화
- Encapsulation(캡슐화)
  - 객체의 상태와 행위를 하나로 묶어 외부에서의 접근을 제한함으로써 객체를 보호
- **Inheritance(상속)**
  - 이미 있는 클래스로 새로운 클래스를 생성하는 과정

Class

- User-defined data type

```
#1단계 : 클래스 생성
class  Car :
    color = ""
    speed = 0

    def upSpeed(self, value) :
        self.speed += value

    def downSpeed(self, value) :
        self.speed -= value

#2단계 : 객체 생성 (인스턴스 생성)
mycar1=Car()

#3단계 : 필드나 메소드 사용
mycar1.speed = 0
mycar1.color = "파랑"
mycar1.upSpeed(30)

print(f"색상 = {mycar1.color}, 속도 = {mycar1.speed}")
 # 색상 = 파랑, 속도 = 30
```

Constructor(생성자)

- **Instance(인스턴스)**를 생성하면 호출되는 메소드
  - mycar1=Car()
    mycar1.speed = 0
    mycar1.color = "파랑"

- mycar1 = Car()로 인스턴스를 생성한 뒤, mycar1.speed = 0과 mycar1.color = '파랑'으로 이미 생성된 인스턴스의 필드 값을 따로 변경할 수 있다. 하지만 __init__ 생성자를 사용하면 인스턴스를 생성하는 동시에 필드 값을 초기화할 수 있다.

```
class Car:
    color = ''
    speed = 0

    def __init__(self):
        self.color = "빨강"
        self.speed = 0

myCar1 = Car()
```

- ⇒ Instance를 생성하면 자동으로 Constructor가 호출
- **Parameter(매개변수)**가 있는 Constructor가 있을 수 있음

```
class Car:
    color = ''
    speed = 0

    def __init__(self, value1, value2):
        self.color = value1
        self.speed = value2

## upSpeed(), downSpeed() 함수

myCar1 = Car("빨강", 30)
print(f"색상 = {myCar1.color}, 속도 = {myCar1.speed}")
```

- >> 색상 = 빨강, 속도 = 30

```
class ~~:


### ~~ 생략
def getName(self):
                return self.name
def getSpeed(self):
                return self.speed


print(car1.getName(), car1.getSpeed())
#=> 이름과 스피드를 반환하는 함수를 메소드로 만들 수 있음
```

Instance variable(인스턴스 변수)&Class variable(클래스 변수)

- instance variable : class 안에 있지만, instance마다 변하게 됨.
- Class variable : class 안에 공간이 할당된 변수

```
class Car:
    color = '' # 클래스 변수
    speed = 0 # 클래스 변수
    count = 0 # 클래스 변수

    def __init__(self):
        self.speed = 0 # self로 정의한 인스턴스 변수
        Car.count += 1

myCar1, myCar2 = None, None

myCar1 = Car()
myCar1.speed = 30
print(myCar1.speed, Car.count) # 30, 1

myCar2 = Car()
myCar2.speed = 60
print(myCar2.speed, myCar2.count) # 60, 2
```

- ⇒ Car.count, myCar2.count 모두 접근할 수 있음

Inheritance

- 기존 class의 필드와 메소드를 그대로 물려받는 새로운 class

```
class 서브 클래스(슈퍼 클래스):
//서브 클래스의 내용을 코딩
```

- ex> 자동차 클래스를 만들고, 승용차와 이를 상속받도록 할 수 있음

```
class Car:
    speed = 0

    def upSpeed(self, value):
        self.speed += value

    def downSpeed(self, value):
        self.speed -= value
```

```
class Sedan(Car):
    seatNum = 0

    def getSeatNum(self):
        return self.seatNum
```

- **Method(메서드)** Overriding
  - 슈퍼 Class의 메소드를 서브 Class에서 재정의 하는 것

```
class Car:
    speed = 0

    def upSpeed(self, value):
        self.speed += value

    def downSpeed(self, value):
        self.speed -= value

class Sedan(Car):
    seatNum = 0

    def upSpeed(self, value):
        self.speed += value

        if self.speed>150:
            self.speed = 150

        # 최대속도라는 새로운 조건을 추가할 수 있음

    def getSeatNum(self):
        return self.seatNum
```

Operator Overloading(연산자 오버로딩)

- 사용자 정의 클래스에서 +, -, *, == 같은 연산자를 직접 정의해서 사용하는 것

-

메소드	연산자	사용 예
__add__(self, other)	+ (이항)	A + B, A += B
__sub__(self, other)	- (이항)	A - B, A -= B
__mul__(self, other)	*	A * B, A *= B
__truediv__(self, other)	/	A / B, A /= B (3 이상 지원, 그 이하는 버전에서는 __div__가 사용)
__floordiv__(self, other)	//	A // B, A //= B
__mod__(self, other)	%	A % B, A %= B
__divmod__(self, other)	divmod()	divmod(A, B)
__pow__(self, other[, modulo])	pow(), **	pow(A, B), A ** B
__lshift__(self, other	<<	A << B, A <<= B

- __add__를 메소드로 정의하면 + 연산자를 사용할 수 있음

```
class Vector:
    def __init__(self, x, y):
        self.x, self.y = x, y

    def __add__(self, other):
        return Vector(self.x + other.x, self.y + other.y)

    def get_vector(self):
        return self.x, self.y

v1 = Vector(1, 2)
v2 = Vector(3, 4)
v3 = v1.__add__(v2)
print(v3.get_vector())
v3 = v1 + v2
print(v3.get_vector())
```

- \>(4, 6) (4, 6)
- ⇒ v1 + v2가 자동으로 **add** 메서드를 호출해 (4,6)을 반환

Polymorphism(다형성)

- inheritance 관계 내의 다른 Class들의 instance들이 같은 멤버 함수 호출(같은 방법)에 대해 각각 다르게 반응하도록 하는 기능

```
class Animal:
    def __init__(self, name):
        self.name = name
    def speak(self):
        return '알 수 없음'

class Dog(Animal):
    def speak(self):
        return '멍멍!'

class Cat(Animal):
    def speak(self):
        return '야옹!'

animalList = [Dog('dog1'),
              Dog('dog2'),
              Cat('cat1')]

for a in animalList:
    print (a.name + ': ' + a.speak())
```

← 프로그래밍과 문제해결 · CSED101 목차로